

URBANBLADE

Documentación Técnica

Sistema de gestión de barbería con módulo de analítica avanzada integrado

Aplicación web (Laravel 12 + MongoDB) con módulo de analítica avanzada integrado

Equipo UrbanBlade

07 de agosto de 2026

Contenido

Contenido.....	2
1. Arquitectura general	3
2. Aplicación web (barber/)	3
2.1 Stack tecnológico	3
Paquetes clave	3
2.2 Infraestructura Docker.....	3
2.3 Roles y permisos	4
2.4 Módulos de dominio	4
Máquina de estados de citas	5
Pagos y fidelización	5
Tienda de productos	5
Muro social	5
Notificaciones	5
Campañas de marketing	5
Tarjeta de socio.....	5
Reportes con gráficas.....	6
2.5 Base de datos (MongoDB)	6
2.6 Rutas	7
2.7 Estado de pruebas.....	7
3. Módulo de analítica avanzada	8
3.1 Qué es y cómo se accede	8
3.2 Cómo está organizada la información	8
3.3 De dónde salen los hallazgos	8
3.4 Vista por pestañas.....	8
4. Integración entre la app y el módulo de analítica	9

1. Arquitectura general

UrbanBlade es un único proyecto (aplicación web Laravel) que corre sobre MongoDB Atlas e incluye, integrado en la misma base de código, un módulo de analítica avanzada — antes desarrollado como un proyecto aparte y ya incorporado por completo a la aplicación principal.

Proyecto	Repositorio	Función
barber/	KikeGonRam/barber (rama feature/mongodb-migration)	Aplicación web Laravel — la app en producción
analítica/	KikeGonRam/barber (mismo repo, ya integrado)	Módulo de analítica avanzada, accesible en /analítica dentro de la misma app

 ESPACIO PARA CAPTURA DE PANTALLA

Diagrama de arquitectura general del sistema (opcional: capturado de una herramienta de diagramación).

2. Aplicación web (barber/)

2.1 Stack tecnológico

Backend: PHP 8.2+, Laravel 12. Base de datos: MongoDB (paquete mongodb/laravel-mongodb) — sin MySQL. Frontend: TailwindCSS 3/4 + Alpine.js 3 + Vite, renderizado en servidor con Blade (sin framework SPA).

Paquetes clave

- spatie/laravel-permission — roles y permisos, adaptado a MongoDB
- barryvdh/laravel-dompdf — generación de PDF (facturas, recibos, tarjeta de socio)
- endroid/qrcode — códigos QR de la tarjeta de membresía
- stripe/stripe-php — pagos con tarjeta, con webhook dedicado
- Twilio — integrado a mano vía `Http::withBasicAuth()` (sin SDK oficial), para SMS/WhatsApp
- spatie/laravel-activitylog — bitácora de acciones del sistema

2.2 Infraestructura Docker

docker-compose.yml define 6 servicios sobre una imagen compartida (barber-app):

Servicio	Función
app	PHP-FPM — procesa las peticiones
web	Nginx (puerto 8000 expuesto al host)
worker	queue:work — procesa colas (correos, notificaciones, PDFs)
scheduler	schedule:work — tareas programadas (no-show automático, campañas, resúmenes diarios)
redis	Caché y backend de colas
mailpit	Capturador de correo para desarrollo (UI :8025 / SMTP :1025)

Todos los servicios comparten un volumen vendor_data para no reinstalar dependencias PHP por contenedor.

 ESPACIO PARA CAPTURA DE PANTALLA

Salida de "docker compose ps" mostrando los 6 contenedores en estado healthy.

2.3 Roles y permisos

Cuatro roles: administrador, recepcionista, barbero, cliente.

Nota técnica: MongoDB no soporta las tablas pivote (MorphToMany) que usa Spatie Permission por defecto. Además del paquete, se mantiene un campo role_id embebido directamente en el documento de cada usuario, y la protección de rutas usa un middleware propio role.custom:<roles> en vez del role: estándar de Spatie.

Permisos granulares por encima del rol: citas.gestionar, pagos.gestionar, inventario.ver/gestionar, clientes.gestionar, reportes.ver, servicios.gestionar, usuarios.gestionar, barberos.gestionar, configuracion.gestionar, logs.ver. El administrador tiene todos; la recepcionista tiene el subconjunto operativo (citas/pagos/clientes/inventario).

Verificación de cuenta: el registro ya no envía un enlace firmado (flujo por defecto de Laravel Breeze). En su lugar, User::sendEmailVerificationNotification() está sobrescrito para generar un código numérico de 6 dígitos con 10 minutos de vigencia (campos verification_code / verification_code_expires_at en el propio documento del usuario), enviado con EmailVerificationCodeNotification y validado por VerifyEmailCodeController — sin exponer un token en la URL.

2.4 Módulos de dominio

Máquina de estados de citas

Gestionada por AppointmentStatusService:

Estado actual	Puede pasar a
pendiente	confirmada, cancelada
confirmada	en_proceso, completada, no_asistio, cancelada
en_proceso	completada, cancelada
completada / cancelada / no_asistio	(estados terminales, sin más transiciones)

Regla de negocio: el cobro solo se habilita desde confirmada, en_proceso o completada — nunca desde una cita pendiente sin aprobar. El barbero aprueba/rechaza sus propias citas; el cliente solo puede cancelar. Un job programado marca automáticamente como no_asistio las citas confirmadas cuyo horario ya pasó.

Pagos y fidelización

Cada cita completada genera un Payment (monto, propina, método) y una LoyaltyTransaction (puntos ganados). El nivel del cliente (nuevo → regular → vip → leyenda) se recalcula por umbrales de citas completadas. Existe integración con Stripe para pago con tarjeta, con webhook dedicado excluido de CSRF.

Tienda de productos

Catálogo (Product, con tipo = uso_interno o venta), carrito (CartService) y pedidos (Order, OrderService). Un pedido puede ser de dos tipos: cita (add-on comprado dentro de la reserva) o tienda (compra suelta). La entrega se gestiona desde una bandeja de pedidos para recepción/administración, que notifica al cliente al marcar como entregado.

Muro social

Los barberos publican trabajos (Work + WorkImage, portafolio), que los clientes pueden comentar (Comment) y reaccionar (Reaction). Sirve como escaparate del trabajo de cada barbero.

Notificaciones

Multicanal: correo (con plantilla de marca compartida), database (centro de notificaciones in-app) y un canal Twilio propio (SMS/WhatsApp; se simula/registra en log si no hay credenciales configuradas). Eventos cubiertos: confirmación/cancelación de cita (con adjunto .ics y enlace a Google Calendar), recibo de pago con factura PDF adjunta, solicitud de reseña tras completar el servicio, alerta de stock bajo al administrador, resumen diario del negocio, y entrega de pedido.

Campañas de marketing

El módulo Campaign permite enviar promociones ahora o programarlas, con tracking de apertura (/t/o/{campaign}/{user}) y clic (/t/c/{campaign}/{user}) vía píxel/redirección, y métricas agregadas en la UI de campañas. Los clientes pueden optar por no recibir promociones.

Tarjeta de socio

Tarjeta de membresía dinámica según nivel de lealtad, con volteo para mostrar un QR de identificación, animación de confeti al subir de nivel, y descarga en PDF.

 ESPACIO PARA CAPTURA DE PANTALLA

Panel de administración mostrando el catálogo de servicios/productos.

Reportes con gráficas

El centro de reportes (`ReportService::reportByType()`) ya no exporta solo tablas: cada tipo de reporte (ingresos, citas, inventario, clientes) calcula también un bloque chart (labels/values) que se reutiliza en tres salidas — una gráfica Chart.js en la vista previa web, un gráfico de barras nativo embebido en el Excel exportado (GenericReportExport, vía PhpSpreadsheet/WithCharts) y un gráfico dibujado en HTML/CSS puro dentro del PDF (DomPDF no ejecuta JavaScript, por lo que no puede usar Chart.js), manteniendo la paleta de marca (dorado #D4AF37 sobre fondo oscuro) en las tres.

2.5 Base de datos (MongoDB)

17 seeders orquestados por DatabaseSeeder.php en orden estricto de dependencias, cada uno mapeado a una colección:

roles/permissions → barbershop_settings → services → products → usuarios (administrador, recepcionista, barbers, clients) → barber_schedules → appointments → payments → loyalty_transactions → orders → works → work_images → comments → reactions.

Dataset de referencia: 1 administrador + 1 recepcionista + 50 barberos + 1500 clientes, ~112,000 citas históricas (2024 → semana actual, con la regla de que ninguna cita futura puede estar en un estado distinto de pendiente), ~19,000 pedidos de tienda, ~200 publicaciones sociales con comentarios y reacciones.

 ESPACIO PARA CAPTURA DE PANTALLA

Vista de las colecciones en MongoDB Compass / Atlas con sus conteos de documentos.

2.6 Rutas

- routes/web.php: rutas públicas (landing, equipo, servicios, tracking de campañas, chatbot) + grupos protegidos por auth, role.custom:administrador, role.custom:administrador,recepcionista, role.custom:cliente (prefijo /cliente) y role.custom:barbero (prefijo /barbero).
- routes/api.php: webhook de Stripe (sin CSRF), y todo bajo prefix('v1') — endpoints públicos (auth, catálogo, chatbot), luego un grupo protegido por autenticación móvil (mobile.auth), con un subgrupo admin + role.custom:administrador para endpoints administrativos.
- Página pública (/): siete secciones — Inicio, Servicios, Proceso, Maestros, Ubicación, Acceso y Reservar. La sección Ubicación incluye un mapa real embebido (iframe de OpenStreetMap centrado en la dirección configurada) y un enlace a Google Maps, en vez de un marcador estático. Los datos de servicios, barberos y estadísticas globales que alimentan esta página se cachean 5 minutos (Cache::remember en routes/web.php) para no recalcularlos en cada visita anónima.

2.7 Estado de pruebas

El proyecto no cuenta actualmente con un directorio tests/ ni suite de pruebas automatizadas activa. Cualquier verificación de cambios se hace manualmente (navegador + revisión de logs de Docker).

3. Módulo de analítica avanzada

3.1 Qué es y cómo se accede

Es una sección más de la aplicación Laravel, no un proyecto aparte: se accede desde el menú (Analítica, ruta /analitica) y la resuelve AnalyticsController, que arma la pantalla según el rol de quien la visita — mismo patrón de autenticación y permisos que el resto del sistema.

3.2 Cómo está organizada la información

Los hallazgos se agrupan por intención de negocio, no por tema académico, para que cualquier usuario entienda de un vistazo qué puede hacer con cada dato:

- Resumen ejecutivo — el pulso general del negocio.
- Operación y equipo — agenda, ventas e inventario.
- Clientes y ventas — quiénes vuelven, quiénes podrían dejar de venir, qué se vende más.
- Predicción y cancelaciones — estimaciones de demanda y riesgo de cancelación.

Cada rol ve solo el recorte que le corresponde (AnalyticsInsightService::forAdmin() / forReception() / forBarber() / forClient()) — un barbero, por ejemplo, nunca ve los insights privados de otro barbero.

3.3 De dónde salen los hallazgos

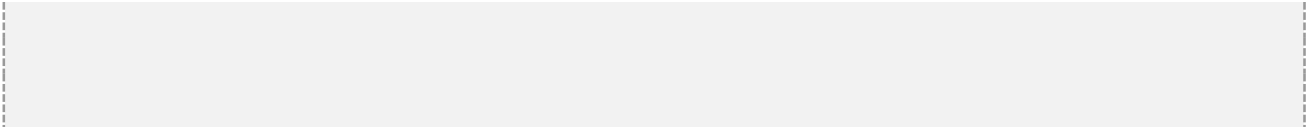
Cada tarjeta que se muestra en pantalla es un documento ya calculado en la colección analytics_insights (modelo AnalyticsInsight): título, mensaje en español sencillo, un valor destacado y a qué rol(es) le corresponde. Laravel solo lee esta colección — no hace cálculo pesado en cada request — y AnalyticsInsightService cachea el resultado 10 minutos por rol para no golpear MongoDB en cada carga de pantalla.

Regla importante: los estados válidos de una cita son 6 (pendiente, confirmada, en_proceso, completada, cancelada, no_asistio) y el ingreso de una cita es siempre su precio_cobrado, nunca cantidad × precio — cualquier cálculo de negocio en el proyecto respeta estas dos reglas para que los números cuadren entre módulos.

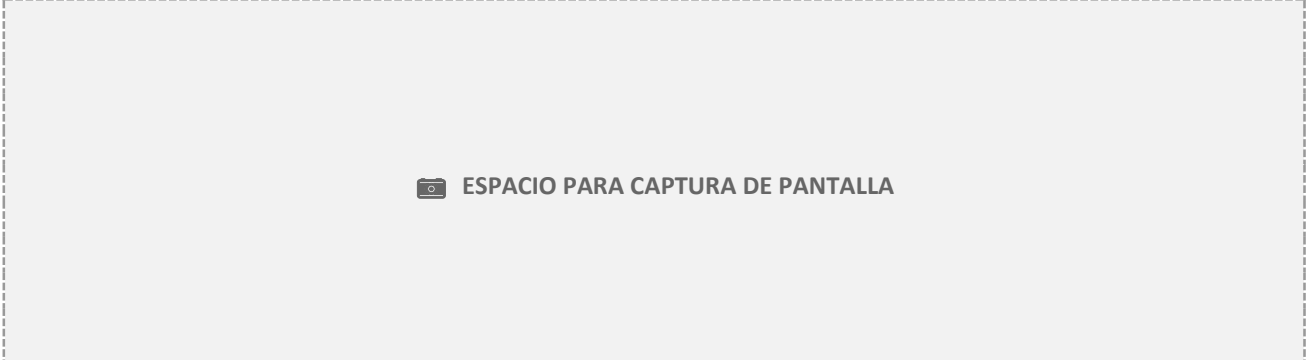
3.4 Vista por pestañas

La vista (resources/views/analytics/index.blade.php) organiza las secciones de la 3.2 en pestañas dentro de la misma pantalla, con un acento de color distinto por sección para que sea fácil de escanear visualmente. El componente analytics-insights.blade.php dibuja cada tarjeta de hallazgo de forma consistente sin importar de qué sección venga.

 ESPACIO PARA CAPTURA DE PANTALLA



Página de Analítica — pestaña "Resumen ejecutivo" con los KPIs principales.



 ESPACIO PARA CAPTURA DE PANTALLA

Página de Analítica — pestaña "Clientes y ventas" u "Operación y equipo".

4. Integración entre la app y el módulo de analítica

El módulo de analítica solo lee de la base de datos — nunca escribe en las colecciones operativas (citas, pagos, clientes, etc.), es puramente de consulta. Cualquier cambio de esquema en el resto de la app (nuevos campos, nuevas colecciones, cambio de estados) debe reflejarse en la lógica que llena analytics_insights para que los hallazgos sigan siendo correctos; esto ya ocurrió una vez (la ampliación de 4 a 6 estados de cita, y la incorporación de orders/works/comments/reactions al análisis, antes vacías).